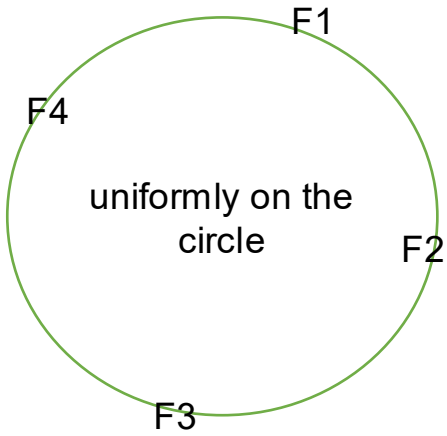
Peer to Peer Networks

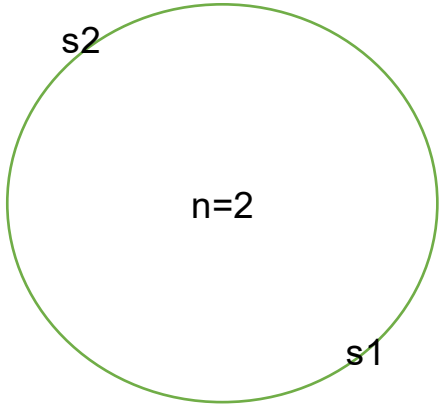
CUHK

P2P is an overlay network

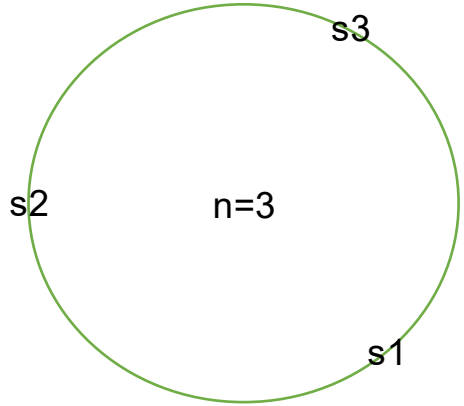
- That is, a logical network
  - E.g., your social network
- Unstructured P2P network
  - P2P = **decentralized** = no one, even the inventor, can dominate
  - E.g., File sharing: a peer can decide which files it hosts (Napster, Gnutella, etc)
    - Finding a file thus requires flooding
- Structured P2P network
  - E.g., File sharing: have to follow rules to hold files
  - E.g., Distributed Hash Table (DHT)
    - Use a variant of consistent hashing to map a file to a particular set of peers
      - **Only K/n keys need to be remapped when the hash table size changes (compared with traditional hashing, every key needs to be remapped)**
      - Standard hashing vs consistent hashing:
        - K mod n (e.g., n = # of nodes)
        - n changes? Need to remap everything

Consistent hashing (hash on an abstract circle)





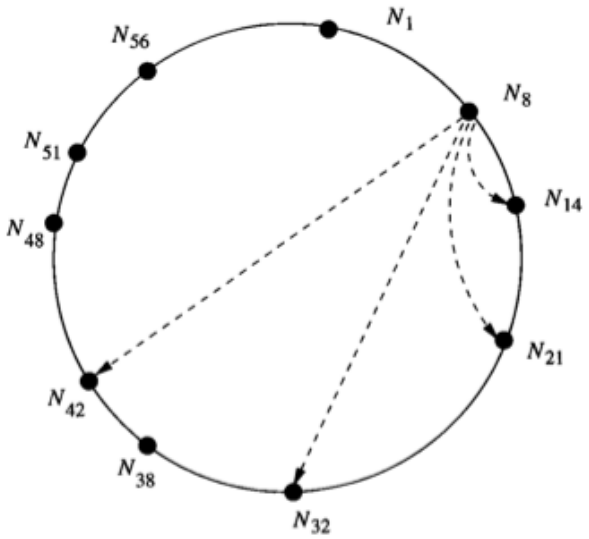
F1 to s1
F2 to s1
F3 to s2
F4 to s2



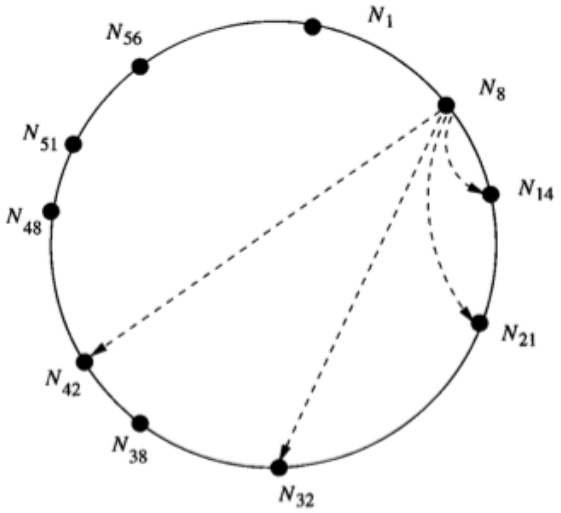
F1 to s3
F2 to s1
F3 to s2
F4 to s3

Chord – DHT (Distributed Hash Table)

- Structured P2P
  - ***A node will store the values for all the keys for which it is responsible***
  - Arrange the peers in a **virtual/logical** "circle"
    - Node N8 is not necessarily close to N14 physically
  - Issues:
    - Searching
    - A peer joins
    - A peer leaves
    - A peer fails



Chord and Links

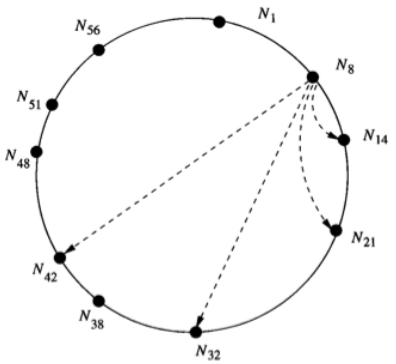


- Each node knows its logical neighbors
- <K, File V>
  - Place V in the lowest node Nj such that j >= h(K)
    - E.g., if h(K) = 44, V goes to N48
    - E.g., if h(K) = 64, V goes to N1
- Basic searching (w/o links)
  - E.g., N8 wants to look up a file whose h(K) = 54
    - N8 forwards its request to N14
    - N14 forwards the request to N21
    - … until N56 says that I am the owner
  - Not efficient
- For efficiency, in addition to neighbors, keep a **finger table** at each node i
  - Each node stores the id of lowest node whose id is at least > i+1, i+2, i+4, i+8, …
  - E.g., finger table of N8 (i.e., i=8)

i=8	+1	+2	+4	+8	+16	+32
Distance	8+1=9	8+2=10	12	16	24	40
Node	N14	N14	N14	N21	N32	N42

Searching

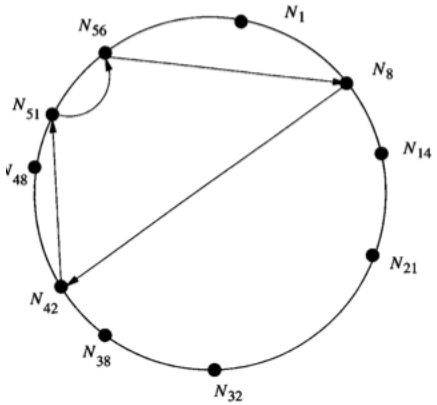
- E.g., N8 wants to search for K
  - Assume h(K) = 54
  - It examines its finger table
    - Found N42 to ask for K



i=8	+1	+2	+4	+8	+16	+32
Distance	8+1=9	8+2=10	12	16	24	40
Node	N14	N14	N14	N21	N32	N42

- N42 finds its successor (N48) ! > 54
- It thus examines its finger table, which is N51

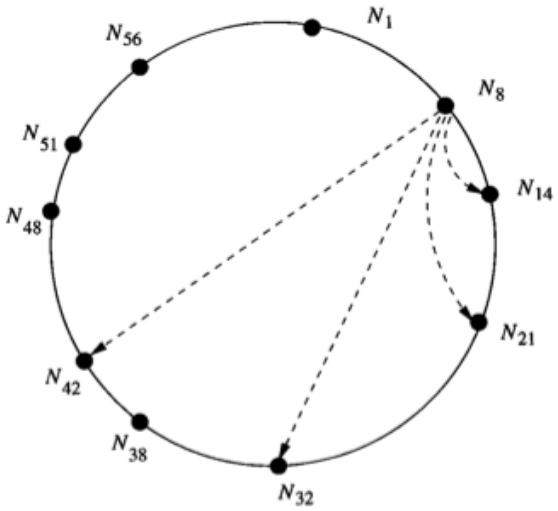
i=42	+1	+2	+4	+8	+16	+32
Distance	42+1=43	42+2=44	48	50	58	74 – 64 =10
Node	N48	N48	N48	N51	N1	N14

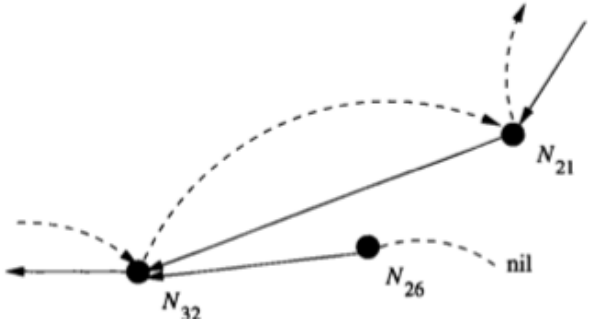


- N51 knows its next is N56, it forwards its requests to N56, which is the lowest node whose ID is > 54.

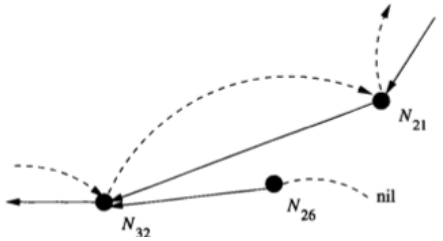
Adding a node Ni

- Ni must know one existing node Nj
- Nj helps Ni to look for the proper peer
- E.g., adding N26
  - A peer can search thru the circle and tell N26's successor is N32
    - N26 sets its successor as N32
- Temporally the circle becomes



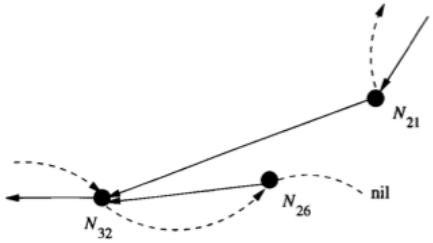


Adding a node N



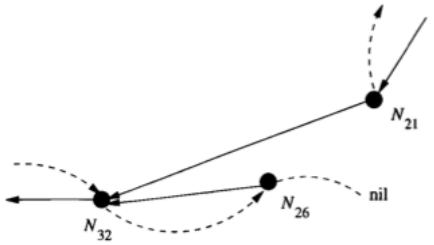
- Nodes periodically carrying out a stabilization process
  - N26 will then trigger stabilization

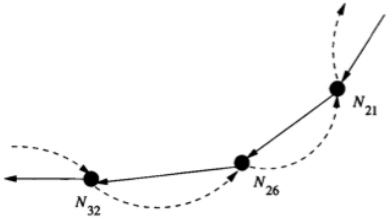
1. Let S be the successor of N
   - N sends a message to S (N32) to ask its predecessor P (N21)
     - If P = N, (normally; no nodes added), skip to Step 4
2. If P (=N21) lies strictly between N (=N26) and S (=N32)
   - then N regards P (=N21) as its successor
   - [In this example, we do nothing because the if-condition is invalid]
3. Let S' be the current successor of N
   - S' = N32 in this ex.
   - If the predecessor of S' is Nil or N (=N26) lies strictly between S' (=N32) and its predecessor (=N21)
     - Then N sends a message to ask S' (=N32) to set its predecessor be N (=N26)
       - In ex. , N32 predecessor is set = N26
4. S' gives N the data that it should be responsible
   - That is, all (K,V) whose keys hash to 22 to 26 are moved from N32 to N26



Adding a node N

- Note: the circle has not stabilized, since N21 and others' finger tables have not updated yet
- For N21,
  - Step 1. S= N32
  - Step 2. N26 lies strictly between N21 and N32
    - Then N21 regards N26 as successor
  - Step 3. Since N26's predecessor is Nil, we make N21 as the predecessor of N26.





When a node leaves

- 1. Notifies its predecessor and successor that it is leaving, so they become each other’s predecessor and successor
- 2. Transfers its data to its successor

- [Finger tables have to be recomputed during a stabilization phase]

More...

- If fail suddenly, data lost!
  - Replication
    - Replicate the data on each node’s predecessor and successor
- When cluster is too large/small, can
  - Split/Merge the cluster into two clusters

Unstructured vs. Structured

- Unstructured:
  - Resilient to high **churn** rate
  - Lookup? Flooding / Gossip search -> slower
- Structured:
  - Pointers to file need to be updated due to churn
  - Lookup is faster
- Consistent hashing is now heavily used in cloud systems
  - E.g., Amazon Dynamo